

Bitwise-Reproducible Reduction Across Cluster Reshape: A Compiler/Fabric ABI

Siddharth Patel, Aman Awasthi, Akshansh Chaurasia, and Rohit Singh

Abstract—Floating-point reductions are not bitwise reproducible under *cluster reshape*: change the worker count or the network and the collective changes its reduction order, and with it the last bits. We measure the premise — reducing identical input *bytes* at fourteen rank counts under PyTorch’s collectives returns fourteen distinct 64-bit roots with spreads reaching 5,120 ulp, while repeats at a fixed shape are bitwise identical. We argue that reduction order is an *interface* concern and treat it as a compiler/fabric ABI: a size-indexed contract fixing the operand binding at every internal node of a canonical full binary tree $\text{FBT}(N)$. No conjunct is ours alone; the conjunction is. The order is fixed at the interface and enforced in hardware by scoreboard-gated operand binding, so a violated assumption fail-stops rather than returning a quietly different number; the result is invariant across network *topologies*, not resource counts alone; the operator is an unmodified IEEE-754 FP64 addition with no wide accumulator; and N is arbitrary rather than a power of two. We instantiate the ABI as SPRS, a topology-parametric SystemVerilog NoC plus compiler, and state two properties — *cross-topology bitwise invariance* (Claim 1) and *presence-gated fail-stop* (Property 1) — connected by a mapping/topology orthogonality proposition. The campaign enumerates the refinement-free face of the compiler’s $\approx 22,000$ -point configuration space exhaustively and RTL-validates every distinct image it produces: of 2,844 images on 39 of 40 instances spanning $N \in [8, 8192]$, $G \in [2, 4096]$ and six topologies, 2,822 settled and every one produced a root bit-identical to the canonical root; 22 did not settle, all but one a fabric watchdog fail-stop, and no run produced a finite root differing from the canonical one. Injected faults establish oracle non-vacuity on 61–65 of 107 verdicts. Because the compiler’s search space cannot reach the bit result, we report what it can reach: a portfolio of 4 of 180 configurations attains the measured optimum on 38 of 39 held-out instances and, refitted with an entire topology withheld, has mean out-of-sample regret 1.0391 on the topology it never saw (95% bootstrap interval $[1.0025, 1.0885]$) — better than a rule that conditions on topology and does see it in training (1.0488). Finally we export the discipline to production collectives in software (*pinned-FBT*), correct the index-range construction we published, and price the repair: bitwise reshape-invariant on all 2,414 cells swept against a negative control that fires, at a geometric mean of $2.57\times$ stock `ncclAllReduce` (provisional, the sweep having run under GPU contention).

Index Terms—Deterministic reduction, collective communication, compiler/fabric co-design, IEEE 754 floating point, network-on-chip, bitwise reproducibility, all-reduce.

I. THE PROBLEM

PRODUCTION all-reduce implementations — ring, recursive halving–doubling, double-binary tree — pick the reduction order that maximises throughput for the observed

topology and worker count [1], [2]. When either changes, so does the order. IEEE 754 addition is non-associative, so the order is part of the answer: for two orders σ, σ' of the same multiset [3],

$$\left| \sum_{\sigma} x_i - \sum_{\sigma'} x_i \right| \leq (N-1)u \sum_i |x_i| + \mathcal{O}(u^2), \quad (1)$$

a bound that is catastrophically loose under cancellation.

The consequence is measured in §VI-A, not asserted: reducing identical input *bytes* at fourteen rank counts under PyTorch’s distributed collectives [4] — same data, same seed, same code, only the shape of the job changing — returns fourteen distinct 64-bit results, while repeated runs at any *fixed* shape are bitwise identical five times out of five. Reshape moves the answer; noise does not. In a training loop the divergence compounds: the reduced gradient becomes the next step’s weights. Bit-for-bit regression testing fails the same way — a code accepted on a development cluster produces different last bits on the production cluster, and the reviewer cannot separate a new bug from the reshape artefact. In regulated deployments, inability to replay a computation bitwise across hardware generations is increasingly disqualifying.

Two sources produce this. *Order-dependence of the reducer*: NCCL selects its (algorithm, protocol) pair at communicator initialisation from a cost model over architecture, node and rank counts and message size [2]; the data are never consulted, so with all randomness suppressed the algorithm remains a deterministic function of $(G, \text{topology})$. *Reassociation inside the reducer*: pipeline overlap, SIMD lanes and message merging are each an implicit reassociation. Both have been attacked before, along one axis at a time (§VIII).

What no line of work states is the invariant *at the interface*, enforced below the software that could violate it. A library flag cannot forbid Source 1, because the library has no authority over what cluster size is launched tomorrow; a fabric guarantee alone cannot forbid Source 2, because the fabric cannot see the tree a scheduler intends. The discipline has to be a contract shared by both. The principle is not new — IEEE 754-2019 already makes reproducibility an obligation on language processors, forbidding reordering and fusion under its *reproducible* attribute [5] — but that obligation binds within one program and says nothing about how many workers run a collective or how they are wired. We extend it to the interface the standard does not reach, and index it by problem size so that it survives a change of deployment.

Contributions. (C1) Reduction order as a size-indexed compiler/fabric ABI, with a structural invariance argument (§II): Claim 1 (cross-topology bitwise invariance) and Property 1 (presence-gated fail-stop), connected by Proposition 1. We

claim the framing, not the difficulty — once operand binding is a function of tree-node identity alone, invariance follows by unwinding definitions, and we say so rather than dressing it as a discovery. **(C2)** A fabric (§III) and compiler (§IV) that jointly realise it, in which topology is a runtime table rather than an RTL artefact and one compiler pass *constructs* the single-writer discipline the argument needs. **(C3)** A hardware campaign at scale and a structural pricing of the ABI (§§V, VI), plus a software export onto production collectives (§VII). What is *not* claimed: interoperability with any external reducer, strict IEEE 754 (§II-F), silicon timing, or novel component algorithms — our eighteen assignment and ten mapping algorithms are prior art [6]–[8]; the observation is that any choice among them inherits invariance by composition.

II. THE REDUCTION ABI

A. The Canonical Tree

Definition 1 (Canonical full binary tree). For $N \geq 1$, $\text{FBT}(N)$ is the heap-indexed binary tree with $2N - 1$ nodes: root at index 0, children of i at $2i + 1$ and $2i + 2$; internal nodes $\{0, \dots, N - 2\}$, leaves $\{N - 1, \dots, 2N - 2\}$. Since $2N - 1$ is odd, every internal node has exactly two children: the single-child edge case is eliminated at the tree level.

Definition 2 (Canonical leaf enumeration). idx assigns index k to the k -th smallest *heap index* in the leaf set of $\text{FBT}(N)$.

Definition 2 needs stating precisely because the obvious paraphrase is wrong. Ascending heap index and left-to-right *visual* order coincide exactly when N is a power of two. Otherwise $\text{FBT}(N)$ is depth-imbalanced and a shallower leaf carries a *smaller* heap index than deeper leaves to its left: for $N = 7$ the heap order is $\langle 6, \dots, 12 \rangle$ while visual order is $\langle 7, \dots, 12, 6 \rangle$. Seven of the sixteen leaf counts we evaluate are of this shape, and at $N = 200$ the two enumerations give roots differing by one ulp. An implementation using visual order implements a *different* contract.

Write π_N for the post-order traversal of $\text{FBT}(N)$ restricted to internal nodes — a pure function of N ; $\pi_4 = \langle 1, 2, 0 \rangle$. The canonical evaluation is

$$R_{\text{canonical}}(N, L) = \text{evaluate}(\text{FBT}(N), L, \text{fadd}_{\text{rtl}}), \quad (2)$$

with L indexed by idx and fadd_{rtl} the combinational semantics of our FP64 adder (§II-F). Equation (2) is the *target* of every SPRS execution on (N, L) .

The canonical order is a constraint, not a tautology. One might suspect that any association order yields one root per N , so agreement across topologies measures nothing. We measured how far that is from true. On the leaf vectors evaluated here, right-to-left sequential summation differs from $R_{\text{canonical}}$ by up to 55 ulp at $N = 8192$, and left-to-right differs at twelve of the sixteen leaf counts. The discriminating power is strongly N -dependent and not monotone: exhaustively at $N = 8$, all 40,320 leaf permutations yield only three distinct roots, 88.6% of them the canonical one, so a uniformly random leaf-order violation is detected with probability 0.114; at $N = 16$, all 9,694,845 in-order tree shapes yield four distinct roots spanning two ulp. Across the sixteen leaf counts

the probability that a sampled reassociation moves the root ranges from 0.08 to 0.90. Two consequences follow and we adopt both: the oracle is non-vacuous, so an agreeing run is evidence; and the rows of the invariance table are not equally informative — at small N agreement is close to free, and the weight of the evidence sits at the large leaf counts.

B. The ABI

Definition 3 (Operand binding). For every internal node v of $\text{FBT}(N)$ the compiler emits exactly one `COMPUTE` on the PE hosting v , with $\text{addr_a} = \text{dmem}(\text{left}(v))$ and $\text{addr_b} = \text{dmem}(\text{right}(v))$, where $\text{dmem}(\cdot)$ is a per-PE address assigned by the allocator. The binding is a pure function of v .

Definition 4 (Leaf binding). For every leaf k , the `GENERATE` on the PE hosting k writes $L[\text{idx}(k)]$ into the DMEM slot bound to k .

The fabric executes the two disciplines symmetrically: `COMPUTE` reads both DMEM slots at one clock edge, scoreboard-gated, and writes $\text{fadd}_{\text{rtl}}(\cdot, \cdot)$; `GENERATE` delivers $L[\text{idx}(k)]$ with the scoreboard set. Neither discipline references G , topology, assignment α , mapping μ , refinement r , or timing. These are left free precisely so that schedulers may keep optimising without touching the contract, and they are what the campaign of §V searches over.

C. Assumptions

Fifteen assumptions carry the two properties: five *compiler obligations* (C^*), discharged by the compiler or the user, and ten *fabric invariants* (F^*), established by RTL inspection. Eleven are hypotheses of Claim 1; the four marked $\dagger$ are liveness invariants that can stop a run but cannot alter a root the fabric does produce, and are the hypotheses of Property 1 instead. Appendix A states all fifteen. The partition is not cosmetic: the C^* obligations are post-conditions on the emitted image, checkable without executing anything, while the F^* invariants are properties of the fabric independent of any compilation — so a failure of Claim 1 is diagnosable as a compiler bug or an RTL bug, not both.

Three conditions a reader might expect are absent. *Compilation determinism* is not a hypothesis: Claim 1 quantifies over every image the compiler can emit, and a statement true of all of them does not need the compiler to be a function of its seed. *Partition connectedness* is not either, although 2,061 of the RTL-validated configurations (73.0%) have partitions that are not connected subtrees: operand binding is by node identity and every cross-PE child gets a dedicated receive slot, so the emitted binding is $(\text{left}(v), \text{right}(v))$ for *any* assignment — checked statically on 1,010,895 `COMPUTE` and 731,982 `SEND` instructions across 681 configurations, with zero violations. What disconnection costs is DMEM occupancy, and that is what C-mem bounds. The *identity slot* $\text{DMEM}[0]$ is reserved but unread here, since $\text{FBT}(N)$ has no single-child internal node.

D. Claim 1 and the Fail-Stop Property

Claim 1 (Cross-topology bitwise invariance). *Let $N \in \mathbb{N}$, let $L \in (\text{bits}_{64})^N$ be a fixed leaf-value vector, and let $\mathcal{T}$ be the family of flat direct-network topologies realisable by adjacency tables consistent with `noc_system.sv`. Assume **C-sw**, **C-bind**, **C-leaf**, **C-mem** and **F-sb-mono**, **F-sb-gate**, **F-fw**, **F-inorder**, **F-gci-haz**, **F-arb**, **F-fadd**. Let G be a worker count and $\tau \in \mathcal{T}$ a topology for which the compiler emits an image satisfying **C-mem**; let α, μ, r be any valid assignment, mapping and refinement, and s the emitted per-PE schedule. If the run terminates normally — `all_done` asserted, no `P_ERROR`, no watchdog — then the observed 64-bit root, the result of the root PE's final `COMPUTE`, satisfies*

$$R_{\text{observed}}(N, L, G, \tau, \alpha, \mu, r, s) = R_{\text{canonical}}(N, L). \quad (3)$$

Three features of the statement bind a reimplementer. *First*, **C-mem** is a compilability precondition, not a decoration: the claim asserts nothing about a configuration whose image exceeds the per-PE DMEM budget or whose router count exceeds what the adjacency interface can address. One instance in our suite is in that position, and it is also the one that never settles (§IX). *Second*, the claim is partial-correctness; liveness is Property 1's business, which is exactly why **C-route**, **F-credit**, **F-misroute** and **F-watchdog** are not hypotheses here. *Third*, the observable is the root PE's final `COMPUTE` result, not the root DMEM slot, which no emitted instruction ever reads back.

Property 1 (Presence-gated fail-stop). *Suppose an assumption violation causes some `COMPUTE` to name a DMEM address whose presence bit is never set — a misrouted and dropped packet (**F-misroute**), a lost credit (**F-credit**), an unreachable destination (**C-route**), or an operand bound to a slot no agent writes (**C-bind**). Then no instruction retires on the affected PE, **F-watchdog** fires after `WATCHDOG_MAX` = 20,000 consecutive non-retiring cycles, and the PE enters `P_ERROR` recording `error_pc` and `error_state`; `all_done` is never asserted and no root is produced. An invalid opcode raises `P_ERROR` immediately.*

The converse bound is equally explicit, because the property is easy to over-read. The fabric checks presence, not identity: a `COMPUTE` whose operands are both present executes, whatever they contain. Violations leaving every read address present — an operand misbound to a live slot, a corrupted leaf value, a duplicate write to a receive buffer, an over-budget image in which two addresses alias onto one written slot, or a violation of any of the seven safety **F-*** invariants — terminate normally with a wrong root. The fabric does not detect these and we do not claim it does; they are discharged by the compile-time post-condition on the emitted image and by comparison of the final root against the oracle.

How the hypotheses carry the claim. Four steps, each of the eleven used at least once. **Addresses** (**C-bind**, **C-mem**): `build_schedule` binds a `COMPUTE`'s operands from an address map keyed on node identity, and every cross-PE child holds a dedicated receive slot, so the emitted binding is $(\text{left}(v), \text{right}(v))$ under any assignment; **C-mem** is what

promotes that address-level statement to a value-level one, since the fabric truncates addresses and distinct addresses stay distinct only while the image fits DMEM. **Uniqueness** (**C-sw**, **F-sb-mono**): $\text{FBT}(N)$ is a tree, so each node has one consumer; the allocator therefore gives each receive buffer exactly one writer, and its scoreboard bit transitions $0 \rightarrow 1$ exactly once per program. **Values** (**F-sb-gate**, **F-fw**, **F-inorder**, **F-gci-haz**, **F-arb**): every address a `COMPUTE` reads is a receive buffer or a local temporary. For the first, **F-sb-gate** holds the instruction in DE until both bits are set, so at the EX edge both slots hold their committed remote writes, with **F-fw** closing the BRAM read-first race. For the second the scoreboard is silent — a recycled slot's bit is already set — and need not speak: under **F-inorder** the overwriting `COMPUTE` samples its operands strictly before writeback, and the one writer program order does not sequence, an in-flight **GENERATE**, is covered by **F-gci-haz** and its retry latch **F-arb**. **Function** (**F-fadd**): the committed value is `faddrtl` of those stable inputs, a pure function. Composing, every internal DMEM value at termination is `faddrtl` applied to v 's two children — with leaves supplied by **C-leaf** — so the root PE's final `COMPUTE` returns $\text{evaluate}(\text{FBT}(N), L, \text{fadd}_{\text{rtl}}) = R_{\text{canonical}}(N, L)$, in which no free axis appears.

We label these a *claim* and a *property* rather than theorems because the supporting argument proceeds by RTL inspection, a compiler post-condition, end-to-end root comparison and mutation testing, not by formal proof (§IX, L12).

E. Mapping/Topology Orthogonality

Of the axes Claim 1 quantifies over, (G, α) is where the substantive work lives: changing G changes the number of PEs and changing α changes which PE hosts which node. The $(\text{topology}, \mu, \rho, s)$ slice admits a sharper statement.

Proposition 1 (Mapping/topology orthogonality). *Fix N, L, α, r and the **FADD** unit. For any two valid mappings μ_1, μ_2 and any two valid routing tables ρ_1, ρ_2 for (possibly different) topologies in $\mathcal{T}$,*

$$R_{\text{observed}}(\mu_1, \rho_1) = R_{\text{observed}}(\mu_2, \rho_2) = R_{\text{canonical}}(N, L). \quad (4)$$

Structural argument. By inspection of `build_schedule` and `generate_instructions`: a `COMPUTE`'s $(\text{addr}_a, \text{addr}_b, \text{dest})$ triple depends only on α and the DMEM allocator's output, neither of which reads μ or the routing tables. A `SEND`'s $(\text{dest_gpu}, \text{target_addr}, \text{link})$ triple does depend on μ and on routing, but those determine only *where* data flows, not the operand binding at the receiving PE. Under **F-sb-gate** the receiving `COMPUTE` reads DMEM by address, not by arrival time, port or VC. Hence R_{observed} is independent of (μ, ρ) . $\square$

Proposition 1 is the cleanest statement of what the ABI buys: the two things that change most obviously under cluster reshape are structurally orthogonal to the bit result. This is a consequence of the code path that emits the instructions, not a hypothesis to be validated empirically.

F. FADD Policy

Our adder (`fp64_add.sv`, with a bit-exact Python mirror) implements a *deterministic subset* of IEEE 754-2019 binary64 addition, which we specify fully as *FTZ+RNE*: subnormal inputs flushed to ± 0 preserving sign; round-to-nearest-even only, with no rounding-mode register; post-round subnormals flushed likewise; $x + (-x) = +0$ and $(-0) + (-0) = +0$ (a documented deviation from §6.3 of the standard); a quieted qNaN whose sign and payload are taken from a when a is NaN and from b otherwise; and the usual infinity rules with $\pm\infty + \mp\infty$ returning the canonical qNaN. Invariance depends only on `fadd_rtl` being a pure function of its inputs, not on any particular IEEE 754 feature; a subnormal-aware alternative inherits Claim 1 by construction. The subset is checked rather than asserted: the shipped adder shows zero disagreements with correctly-rounded IEEE 754 addition over 4,644,000 vectors inside the documented scope. The adder we first submitted did not meet the RNE clause on effective subtraction; §IX (L13) reports why neither the campaign nor the mirror could have found that, which is a finding about the evaluation method rather than an erratum.

III. THE FABRIC

The fabric realises the F^* invariants. It supports up to 4096 PEs in a single Verilog instantiation, and the guiding principle is that *topology is data, not RTL*.

Topology as a runtime parameter. Two compiler-loaded per-router tables parameterise the network: an *adjacency table* mapping (router, port) $\rightarrow$ (target router, target port) | `ADJ_DISCONNECTED`, and a *routing table* mapping (router, destination) $\rightarrow$ output port. A single `noc_system` module supports the whole family $\mathcal{T}$; port 0 is reserved for the local network interface on every router, the one structural convention independent of topology. Adding a topology requires a Python classmethod that emits the two tables and no Verilog change: the compiler provides six families (linear, ring, 2D mesh, 2D torus, fat-tree, hypercube) with $O(1)$ analytical shortest paths and a BFS fallback. The binding ceiling is the width of the configuration interface rather than a design parameter — adjacency writes carry a 12-bit router id whose top code is the disconnected sentinel, so ids are addressable to 4,094 and a configuration above that aliases silently. That is exactly what defeats one instance of the suite (§IX, L0). Router radix is likewise per-instance (`PORTS_PER_RTR`), ranging 3–66 across the suite; since the crossbar term is $O(P^2)$, a fat-tree router is $20\times$ a mesh router (750,976 against 37,728 storage-equivalent bits, 56% of it crossbar), while per-PE memory is topology-independent at 512×64 DMEM, 65,536 bits per PE. These are RTL-level counts from a research prototype, not synthesis results; we report no timing closure anywhere.

ISA and pipeline. Four opcodes are live: `NOP`; `GENERATE`, which fetches a leaf into `DMEM[dest]` through the GCI interface and sets `scoreboard[dest]`; `COMPUTE`, which reads `DMEM[addr_a]` and `DMEM[addr_b]` scoreboard-gated, writes `fadd_rtl(a, b)` to `DMEM[dest]` and sets its bit; and `SEND`, which enqueues an RDMA-push packet carrying

`DMEM[src]`. Opcodes beyond one reserved slot trap to `P_ERROR`. Each PE runs a three-stage pipeline — `IF` (registered IMEM read, decode), `DE` (scoreboard hazard check, DMEM read issue), `EX` (combinational ALU, writeback or TX enqueue) — with two-level write-to-read forwarding. The ISA is small enough that the determinism-critical mechanisms fit in a two-page RTL inspection, which is the point: the scoreboard is the single synchronisation point across the three agents that write DMEM. Packets are 96-bit single-flit and carry one FP64 value; the VC-steering bit is written as a literal `6'b0` by the only packet-forming assignment in the design, so every packet travels on `VC0` and cycle counts are value-agnostic by construction.

Two mechanisms that are necessary for the value, not merely for liveness. First, the DMEM write arbiter is strict-priority $\text{PIU} > \text{GCI} > \text{FSM}$ with a GCI *retry latch*: when a GCI completion collides with a PIU write, the arbiter drops the GCI write and the latch re-asserts it until accepted. Without the latch the leaf value would be silently lost while PIU still set the scoreboard bit, and the next `COMPUTE` would read a wrong leaf and produce a wrong root with no diagnostic. Second, the in-flight `GENERATE` interlock stalls any `COMPUTE` or `SEND` naming a pending destination. The RTL comment calls it belt-and-braces with the scoreboard; it is not. 35.4% of `GENERATES` (29,097 of 82,084) target a *recycled* slot whose scoreboard bit is already set by the previous writer, so the scoreboard cannot stall their consumer and the interlock is their sole protection.

Fail-stop discipline. On each drained packet the PIU compares the header's `dest_gpu_id` against the local PE; on mismatch it drops the packet and returns credit rather than writing it (F-misroute). A bad routing table therefore leaves the destination's scoreboard bit clear, its dependent `COMPUTE` stalls, and the watchdog raises `P_ERROR` after 20,000 cycles. Silent corruption is converted into a liveness failure with a recorded program counter — which is Property 1.

IV. THE COMPILER

SPRS compiles $(N, G, \text{topology})$ into per-PE IMEM images plus routing and adjacency tables. Every image reported here was produced by five stages: the composite lower bound Ω (S0, a reported metric rather than a construction step, §V); `FBT(N)` instantiation (S1); assignment α of tree nodes to workers, from 18 algorithms (S2); mapping μ of workers to physical PEs, from 10 (S3); and code emission with cycle-accurate NOP padding (S6). Three further passes exist in the released compiler and produced none of these results: connectivity repair (S2.5), memory enforcement (S5), and refinement (S4, twelve base methods chainable to 122 configurations), fixed at the identity throughout.

One pass is load-bearing. The DMEM allocator walks the Sethi–Ullman-refined post-order. It reserves address 0 on every PE; at each node v on PE $p = \alpha(v)$ it returns to p 's free list the slot of every child of v already resident on p , then gives v the head of that list or else a fresh address. A final pass gives every cross-PE edge a persistent receive-buffer slot on the parent's PE, above that PE's local high-water mark, never freed. Three

properties of that walk *construct* C-sw rather than assume it: FBT(N) is a tree, so every node has exactly one consumer; post-order visits children before the parent, so a local slot is freed only after its consumer has read it; and receive slots live in a separate never-freed pass, because a scoreboard-gated COMPUTE needs its address stable. An allocator working in pre-order, or freeing before the consumer ran, would break the discipline. The same walk yields C-mem free: peak local occupancy plus receive slots is exactly the number of distinct addresses the image names on p , so the capacity obligation is an $O(N)$ read-off of a quantity the allocator already maintains.

The repair pass that looks load-bearing is not. Twelve of the eighteen assignment algorithms can hand a PE a node set that is not a connected subtree, and 2,061 of the 2,822 validated configurations are of that kind. Operand binding is unaffected (§II-C); what disconnection costs is DMEM. Over 681 configurations, peak per-PE occupancy has median 10 slots on the 244 connected partitions and median 25 (max 491) on the 437 disconnected ones, against the 512-entry budget — a tail margin of 21 slots. Running `repair_connectivity` removes the pressure and destroys the load balance the assignment search exists to produce: it moves a median 50.8% of all tree nodes, inflates peak per-PE node count by a median factor of 1.86 and up to 304×, and leaves 82 of the 437 with more than 512 instructions on a single PE against exactly one before repair. It is not even monotone in the quantity it is invoked for: in 33 of them it *raises* peak DMEM. Connectedness is a correlate of memory pressure, not a guarantee of it, so we report the unrepaired pipeline and check the bound directly.

Mapping barely reaches the image. SEND carries the logical worker id and IMEM is loaded by logical index, so μ reaches the emitted image only through the hop counts the cost model uses to place NOPs: recompiling five representative points under all ten mappings yields one–three distinct images, differing only in NOP padding. Invariance across S3 is therefore close to tautological, and the evidential weight of the campaign sits on the assignment axis and on reshape across $(G, \text{topology})$.

ABI verification. C-sw, C-bind and C-leaf are post-conditions on the emitted IMEM, checkable without executing anything. The campaign path did not run them in the compile loop, so we discharged them afterwards over the released images — 1,010,895 COMPUTE and 731,982 SEND instructions across 681 configurations, zero violations, no out-of-range address, no receive buffer with two writers. We report it as a static audit of the artifact rather than as a compiler feature, because that is what was run.

V. METHODOLOGY

Instance suite. 40 instances span $N \in \{8, 10, 16, 32, 50, 100, 128, 200, 500, 512, 1024, 1500, 2048, 3000, 4096, 8192\}$, $G \in \{2, \dots, 4096\}$, and the six topologies. Multiple $(G, \text{topology})$ points at the same N are what exercises the invariance claim. Leaf values are

$$L[i] = (i+1) \cdot 1000.0 + ((i \cdot 37 + 13) \bmod 100) / 100 + 0.007, \quad (5)$$

chosen so every leaf carries a non-trivial fractional part: each FADD exercises the guard, round and sticky bits rather than

reducing to an exact power-of-two addition, and magnitudes span 10^3 to 8×10^6 so partial sums accumulate over a wide exponent range. We do not claim reordering *always* moves the result — it does not, which is what the mutation protocol below must work around. What the distribution buys is that a fabric/oracle mismatch is a determinism violation rather than an artefact of degenerate inputs.

Campaign protocol. The per-instance configuration space is $18 \times 10 \times 122 = 21,960$ (α, μ, r) triples; simulating it exhaustively would be $\approx 880,000$ xsim runs. We fix refinement at the identity and enumerate the remaining 18×10 face *exhaustively* on every instance — 180 triples per instance, 0.82% of the space, 7,200 compilations of which 6,777 emit an image. Distinct (α, μ) pairs frequently produce identical images up to PE relabelling; a SHA-256 hash over the emitted testbench collapses the 6,777 to 2,920 distinct ones. That is not merely a compute saving: identical images *tautologically* produce identical bit results, so dedup is what makes each remaining run informative. Every distinct image was then queued for xsim — no shortlist of any kind stands between the compiler and the invariance evidence. A top- K -by-makespan shortlist would have missed a median 20% of the true top- K by measured cycles at $K = 15$, which is why we did not use one.

Mutation protocol. Two classes are injected per instance. *Address substitution* repoints `addr_a` of one randomly chosen COMPUTE at a different DMEM address. *Leaf corruption* perturbs one leaf by the smallest single-bit flip that provably moves the canonical root. The escalation in the second is necessary and is the same phenomenon the paper is about: flipping the mantissa LSB frequently falls below the round bit of the running sum, RNE discards it, and a fabric reporting PASS is correct to do so. We raise the flipped bit until the oracle confirms the root has moved, so every counted mutant tests detection rather than rounding. We deliberately do not use operand swap: `fadd_rtl` is commutative at the bit level under FTZ+RNE, so it would be bit-identical and would inflate the null-detection rate.

Metrics. $\text{hw_cycles} = \max_p \text{perf_total_cycles}[p]$ from per-PE counters is primary; the cost model's makespan is a diagnostic only. We also compute a composite lower bound Ω on any schedule on this fabric,

$$\Omega = \max(\text{lb}_{\text{span}}, \text{lb}_{\text{comm}}, \text{lb}_{\text{gather}}, \text{lb}_{\text{bis}}, \min_{K \in [K_{\min}, K_{\max}]} \max(\text{lb}_{\text{gen}}(K), \text{lb}_{\text{instr}}(K))), \quad (6)$$

where K is the number of PEs a schedule populates, $K_{\max} = \min(G, 2N - 1)$. The terms are the zero-communication critical path, GCI serialisation, instruction throughput, a cross-PE hop floor, the root PE's gather path, and link serialisation at the bisection. Two of the six shrink as work spreads over more PEs, so a bound valid against *every* schedule must take the most favourable K ; a maximum there would assert a bound some legal schedule beats. Because Ω bounds non-deterministic schedules too, it is not a cost-of-determinism denominator but an absolute-performance diagnostic. Summary statistics are medians with observed ranges rather than interval estimates: the per-instance samples are not draws from

a population, they enumerate a deterministic configuration space, so a resampling interval would describe our sampling of that space rather than any underlying uncertainty.

The baseline we did not build. The natural metric is the same-fabric overhead η against the best non-deterministic reducer on the identical substrate, obtained by relaxing C-bind. **We did not build it and report no measured η** , and we decline to substitute an analytical one. We built such a model — the same cost constants that produce Ω , applied to a ring and to recursive doubling — and it returns $\hat{\eta} \approx 1$ on every instance. That is not a modelling artefact. COMPUTE reads two DMEM addresses and no more, so every reducer here is at least $\lceil \log_2 N \rceil$ deep — exactly FBT(N)’s depth, and against the best same-fabric alternative SPRS is never deeper on the 39 validated instances and is one level shallower on three. At power-of-two N and G , cutting FBT(N) at level $\log_2 G$ gives G complete subtrees of N/G leaves and $G - 1$ nodes above the cut, which is recursive doubling; the schedules are DAG-isomorphic, verified by construction on all 88 such pairs up to $N = 8192$. Both are powers of two on 31 of the 39 instances, so $\eta = 1$ is a theorem there, not a measurement we failed to take. The model says 1 because 1 is the answer; we decline to quote it all the same, it being the compiler’s own cost model and so circular. On the other 8 instances the isomorphism fails and a baseline would say something; none was built, and we price the ABI structurally instead (§VI-D).

VI. EVALUATION

Vivado xsim 2025.2 on Linux; ten SystemVerilog sources (3,738 lines); a 72-thread Intel Xeon w9-3475X with 503 GB of RAM; RNG seeds pinned at 42; per-simulation wall-clock cap 28,800 s. Every cycle figure comes from a single campaign of 2,822 passing runs consuming 821.5 core-hours at a median of 99 s per testbench, the five largest instances accounting for 90.0% of that and the longest single run taking 5.51 h. An earlier campaign over the same testbenches is used for no reported number: it recorded PE 0’s counter rather than the maximum across PEs, which understates end-to-end latency whenever the root lands elsewhere. That defect moved 50.5% of cycle counts and changed the *minimising configuration set* on 38 of 39 instances, leaving it disjoint from the corrected set on 32, so the measurements were repeated in full rather than corrected in place. Hardware validation covers 39 of the 40 instances (§IX, L0).

A. The Premise, Measured

The ABI is worth its cost only if reshaping a production collective really does move the bits. On a single node with two NVIDIA RTX PRO 6000 Blackwell GPUs over PCIe (no NVLink) and 72 CPU cores we reduced identical input *bytes* at fourteen rank counts from 1 to 64 under PyTorch 2.11.0’s distributed collectives [4], holding data, seed and code fixed so that only the shape of the job changed. Leaf i ’s value is a pure function of i , so reshaping alters who holds which leaf and in what order the partial sums combine, and nothing else.

The fourteen configurations returned fourteen distinct 64-bit results in every one of twelve (leaf count, precision, value-profile) cases; in the six fp64 cases, for which the single-process canonical reduction was also computed, none of the fourteen matched it. Spreads reach 5,120 ulp and a maximum relative spread of 8.1×10^{-13} ; at least 1,023 of the 1,024 elements of the result vector move in every case; pooled over every configuration swept, 78.6% of fp64 configuration pairs and 90.0% of fp32 pairs disagree. Repeated runs at any *fixed* configuration were bitwise identical five times out of five, with no disagreement between ranks inside a run, and an independent re-run under a different bookkeeping harness reproduced 168 of 168 common cells bit for bit. Reshape moves the answer; noise does not.

Three scope conditions belong with the measurement rather than under it. The rank-count sweep above $G=2$ runs on the CPU backend, because NCCL requires one rank per device and this node carries two GPUs. At $G=2$ the NCCL algorithm-by-protocol axis (seven configurations) and the channel-count axis (three) each returned a single root; that is structural rather than an empirical null — at two ranks the cross-rank reduction is one addition per element and no schedule can reassociate it — and we do not generalise it to larger G . At that same partition the CPU and GPU roots differ in all twelve cases, so device placement moves the bits as well as partitioning does. This measures reduction order on one node; it is not a benchmark of any library and no timing claim is drawn from it.

B. Cross-Topology Bitwise Invariance

We state the primary empirical result in the narrowest form the campaign supports. Across the 2,844 distinct ABI-compliant program images obtained by compiling the complete 18×10 grid on the 39 hardware-validated instances — sixteen leaf counts $N \in [8, 8192]$, requested worker counts $G \in [2, 4096]$, six direct-network topologies — 2,822 simulations terminated, and in every one the root PE’s final COMPUTE result was bit-identical to the canonical root $R_{\text{canonical}}(N, L)$ the software oracle predicts for that N . The remaining 22 did not settle; no configuration produced a finite root differing from $R_{\text{canonical}}(N, L)$.

Two scope conditions belong inside that sentence rather than in a footnote. The campaign evaluates one leaf vector per N and the identity refinement throughout; and eight of the sixteen leaf counts are represented by a single requested (G , topology) point, so at those N invariance is witnessed across assignment, mapping and schedule but not across reshape. 2,403 of the 2,822 configurations lie at leaf counts carrying two or more points, and that is where the reshape evidence sits.

Where reshape is exercised it is not marginal. At $N=128$ the same root survives six (G , topology) points from $G=4$ on a linear chain to $G=128$ on a torus — a $32 \times$ change in worker count across five distinct topologies — and at $N=512$ and $N=1024$ it survives six and five points. The 2,844 images are moreover distinct *programs*, not repeated runs of one program: the campaign deduplicates by SHA-256 over the emitted testbench and simulates one representative per hash class, so two images in the count differ structurally,

TABLE I

CROSS-TOPOLOGY BITWISE INVARIANCE. EACH ROW AGGREGATES ALL HW-VALIDATED SPRS CONFIGURATIONS AT THE GIVEN N OVER ALL SAME- N (G , TOPOLOGY) PAIRS AND ALL COMPILER TRIPLES. “DIVERGENT” IS BY CLAIM 1 EXPECTED TO BE ZERO. THE TWO MUTATION COLUMNS COUNT INJECTED MUTANTS THE HARNESS *flagged*; AS DISCUSSED BELOW, A FLAG IS A VALUE MISMATCH FOR LEAF CORRUPTION BUT USUALLY A FAIL-STOP FOR ADDRESS SUBSTITUTION, AND ONLY THE FORMER BEARS ON ORACLE NON-VACUITY.

N	compiled (G , topo)	HW-val. (G , topo)	HW configs (passing)	64-bit root result	Div.	Addr-sub detected	Leaf-corr. detected
8	2	2	28	0x40E1948E978D4FDF	0	6/6	6/6
10	1	1	17	0x40EADBA0A3D70A3E	0	3/3	3/3
16	1	1	21	0x41009A44BC6A7EFA	0	3/3	3/3
32	3	3	102	0x41201D1FCED91687	0	6/6	6/6
50	1	1	25	0x413374911999999A	0	3/3	3/3
100	1	1	33	0x415343B08CCCCCD	0	3/3	3/3
128	6	6	371	0x415F7E8FF9581062	0	6/6	6/6
200	1	1	44	0x41732B4046666667	0	3/3	3/3
500	1	1	98	0x419DDCAB2C000000	0	3/3	3/3
512	6	6	456	0x419F4FA04418938	0	6/6	6/6
1024	5	5	425	0x41BF47D2026872B1	0	2/2	2/2
1500	1	1	104	0x41D0C665F8400000	0	2/2	2/2
2048	4	4	437	0x41DF43E900FBE76D	0	2/2	2/2
3000	1	1	77	0x41F0C4F764200000	0	2/2	2/2
4096	3	3	331	0x41FF41F48085A1CB	0	2/2	2/2
8192	3	2	253	0x421F40FA04027EFA	0	1/1	2/2

“Compiled” counts the distinct (G , topology) points at that N for which an ABI-compliant image was produced; “HW-val.” counts those carried through RTL simulation. The single shortfall is $N=8192$, where `maxp_mod_fat` ($G=4096$, fat-tree) could not be simulated; the reason is not a wall-clock budget (§IX, L0).

not merely in the triple that produced them. Invariance holds simultaneously under reshape and under complete replacement of the assignment and mapping algorithms.

The campaign has since been re-executed from source. All 1,326 passing configurations at $G \leq 64$ — every one of the twenty-five suite instances at that scale, spanning all six topologies — were recompiled and re-simulated under both route-table loading paths (§IX, L0b), and on the 1,306 points that completed under both, the two paths agree with each other and with the oracle on the same 64-bit value; each instance yields exactly one distinct root across all of its pipelines. The released results file records verdicts and cycle counts but not root words, and the RTL re-simulated is the current tree rather than the exact one that produced that file, so this is a re-execution that self-agrees and re-derives the oracle’s prediction — not a bitwise diff against the originally released values. We state it in those terms.

Table I reports the result, with the divergence column zero as Claim 1 predicts. Its rows are not equally informative, for the reason §II-A measures: at the smallest leaf counts a large majority of alternative association orders reproduce the canonical root anyway, so agreement there is close to free and the weight of the evidence sits at the large- N rows.

Beyond the measured range. Invariance at untested G rests on the structural argument, not on induction: operand binding and scoreboard gating remain total functions of N and L as G grows, so adding PEs changes the *location* of FADD operations, not the operand pairs they receive. Adding a PE does introduce new routing paths, VC contention and credit interactions that may move cycle counts. What carries is the bit result; absolute cycle counts at large G are simulation artefacts, not silicon measurements, and we make no absolute

competitiveness claim. Indeed we make no silicon claim at all: we ship Artix-7 bring-up wrappers for $G \in \{2, 4\}$, but synthesising them needs a Vivado VIO IP core and board constraints we do not redistribute, and we hold no board logs. That is a reporting gap rather than a soundness gap — invariance is a statement about bit values, which xsim resolves exactly.

C. The Oracle Is Not Vacuous, and Where It Stops

A zero-divergence column means nothing unless the oracle can report a non-zero one. Reporting the injected-fault campaign requires more care than a single rate. The harness retains one boolean per mutant, and the testbench prints [FAIL] both on a value mismatch and from its `!all_done` branch on any run that fails to complete; the two are not distinguishable in the recorded output. Reconstructing the mutations against regenerated images separates them. Of the 53 address-substitution verdicts, in at least 42 the value comparison never ran: the substituted address is one no agent ever writes — the harness draws a uniformly random address rather than another node’s — so the dependent `COMPUTE` stalls on the presence gate and the flag is a fail-stop, not a detection. Oracle non-vacuity therefore rests on 61–65 of the 107 verdicts — every leaf corruption, plus the address substitutions that happened to reach a live slot — and not on all 107, as we previously reported.

Read the other way round, the same fact is an asset. 43 of the 54 address substitutions fail-stopped instead of corrupting, and in every one we re-simulated the fabric raised `P_ERROR` with a recorded `error_pc` on every PE. That is Property 1’s halt outcome observed under injected fault rather than argued from the RTL, and the split falls exactly where the presence gate

places the boundary: all 54 leaf-corruption mutants terminate normally with a wrong root — one of them one ulp from canonical — and are caught by the value comparison instead. Mutants were injected across all sixteen leaf-count classes, so no row of Table I rests on an oracle that was never exercised.

D. The Cost of the ABI

We report no measured η (§V), and quoting a ratio against a reducer that does not exist would not be a measurement. But the ABI's cost is not an empirical accident to be discovered by benchmarking; it is a consequence of the contract, and the contract is small enough to price analytically. Four constraints separate SPRS from an unconstrained reducer:

- 1) *FBT depth — zero here.* The ABI fixes $\text{FBT}(N)$, so the dependence chain is $\lceil \log_2 N \rceil$ deep regardless of G ; a reducer forming arity- k partial sums would be $\log_k N$ deep, but $k > 2$ is not expressible on this fabric, so no same-fabric reducer is shallower than $\text{FBT}(N)$ and the term is at most zero (§V).
- 2) *No reassociation.* Fixing $(\text{left}(v), \text{right}(v))$ forbids re-grouping operands by locality. An unconstrained reducer may sum PE-local values first and cross the network once; SPRS crosses wherever the tree says to.
- 3) *GCI serialisation.* Consecutive GENERATES on one PE are spaced by `gci_gap` = 6 cycles, so a PE holding $\lceil N/G \rceil$ leaves spends at least $6\lceil N/G \rceil$ cycles producing them. Unlike term 2 this is an artefact of *our* GCI, not of the ABI.
- 4) *Root-gather fan-in.* The root PE must receive from the other $G - 1$ PEs: at least $\lceil \log_2 G \rceil$ merge stages plus roughly half a network diameter. This binds any reducer, deterministic or not.

Term 2 is the price of reproducibility and cannot be engineered away without abandoning the contract; term 1 is that price in a form this fabric cannot charge; term 3 is ours to fix and term 4 is nobody's.

Term 3, measured directly. The GCI floor $6\lceil N/G \rceil$ accounts for at most 30% of the champion makespan on every one of the 7 instances with $N/G \leq 2$, and for at least 43% on every one of the 14 instances with $N/G > 8$, rising to 64–74% on the 5 instances with $N/G \geq 32$ (Spearman $\rho = +0.77$, $n = 39$). The two groups do not overlap. Since $6\lceil N/G \rceil$ assumes perfectly balanced leaf assignment while any real assignment puts at least $\lceil N/G \rceil$ leaves on the busiest PE, these are *lower* bounds on the true share. Leaf-generation serialisation is thus the dominant term at high N/G : on this suite a multi-channel leaf interface is the highest-value modification available to the fabric, and it costs nothing in the ABI.

A prediction we withdraw. An earlier version tested term 3 by banding cycles/ Ω against N/G and read the decline as a successful prediction of a knee at $N/G \in [6, 8]$. That test is circular. Ω equals $6\lceil N/G \rceil$ on 29 of the 39 instances, so the term under test is the denominator; the absolute distance cycles – Ω carries no N/G signal at all ($\rho = +0.09$, $p = 0.58$); and a one-parameter model with no mechanism in it, cycles = $\Omega + 96$, reproduces the published band medians at and above the predicted knee to within 0.13. Nothing is lost:

TABLE II
DISTANCE TO THE COMPOSITE LOWER BOUND Ω BY TOPOLOGY. Ω BOUNDS ANY SCHEDULE ON THIS FABRIC, SO THESE RATIOS UPPER-BOUND WHAT AN UNCONSTRAINED REDUCER COULD HAVE RECOVERED.

Topology	instances	median cyc/ Ω	best
linear	4	1.39	1.35
hypercube	7	1.85	1.55
torus	10	2.26	1.49
mesh	4	2.58	1.62
fat-tree	8	3.40	1.83
ring	6	4.48	2.33
all	39	2.34	1.35

the direct measurement above does not involve Ω and points the same way term 3 does — indeed the opposite way to the withdrawn reading, since GCI serialisation binds *above* the claimed knee, not below it.

Distance to the lower bound. The one absolute statement available is how far the compiler lands from Ω . Because Ω bounds any schedule here, including a non-deterministic one, cycles/ Ω upper-bounds what an ideal unconstrained reducer could have saved: weaker than η , but real and measured. The informative reading is additive rather than multiplicative. The compiler's absolute distance from the bound is near-constant — median 96 cycles, interquartile range [60, 147] — across a range of Ω from 13 to 384, and uncorrelated with N/G ($\rho = +0.09$, $p = 0.58$). The schedule carries a roughly fixed overhead over the analytic floor regardless of problem size, so relative efficiency improves monotonically with scale and reaches 1.35 $\times$ at the largest instance. No instance reaches Ω , which is expected: the bound is assembled from separately-derived constraints and is not claimed tight. Table II is a descriptive per-topology summary and not a controlled contrast, and we draw no topology ordering from it: the six cells differ in median N/G by a factor of 24, the suite contains only two (N, G) points carrying more than one topology, and those two disagree in sign. An earlier version read the column as ordering with bisection width; that reading is substantially N/G in disguise and we withdraw it.

E. Champion Equivalence Classes

Earlier versions of this paper reported a winner distribution: which assignment and which mapping algorithm produced the fastest image on each instance. We withdraw it, because it was not a measurement, and the reason it was not is this paper's own subject.

The campaign deduplicates by program image before simulating, so one simulated row stands for a class of bit-identical programs and the champion of an instance is a *set*. Expanding those classes back over the full 18×10 grid: the minimum measured cycle count is attained by more than one configuration on 38 of 39 instances, by a median of ten configurations and by as many as 90; all ten mapping algorithms attain it on 27 of 39; and exactly one instance has a unique champion. Which member of a tie set the published table named was decided by a minimum over CSV rows in parallel-worker arrival order — that is, by which simulation

host finished first. A paper about scheduling nondeterminism had its winner table ordered by scheduling nondeterminism, and saying so is more useful than quietly re-sorting it: no tie-break repairs the table, because on 38 of 39 instances the tied set contains programs that are bit-identical and therefore cannot be separated by any measurement at all.

What replaces the ranking is stronger, and its data was already in the paper. Proposition 1 says the compiler’s variation space cannot reach the bit result. The campaign shows that same space collapsing structurally before it ever reaches the fabric: of the 7,200 compilations performed, 59.4% produce a program image that another configuration had already produced. The compiler variation space is thus not merely incapable of changing the answer — over most of its extent it does not even yield a different program. The campaign is, from the ABI’s point of view, 2,822 attempts to break Claim 1, every one of which failed; it is not a ranking of algorithms.

F. A Derived Selection Rule

The campaign was built to answer a practitioner’s question — given an instance, which of the 180 configurations should be run? — and champion-class membership answers it in a form invariant to the tie structure above. For a recommended set we report *regret*: the measured cycle count of its best member divided by the instance’s measured optimum, so 1.000 means the set contains a champion.

The rule is a portfolio, not a regime map. Exhaustive search over subsets of the sixteen near-universally evaluable assignment algorithms, minimising mean measured regret and refitted independently in each of 39 leave-one-out folds, returned the *identical* subset in 39 of 39 folds at every size. At size four that subset is DSC, Lukes, Frederickson, ParSub, and on the held-out instance it attains the exact hardware-measured optimum in 38 of 39 folds (0.974), with mean out-of-sample regret 1.0025 and worst case 1.099. It evaluates 4 of the 180 configurations, a median $144.6\times$ reduction in compiler time (205.6 s to 2.78 s per instance). Table III gives it with the smaller budgets. Size four is a knee rather than a budget we picked: a fifth configuration is identical to four decimal places under both protocols, and only at six does held-out regret reach 1.0000.

Conditioning on the instance makes it worse. This is the result we did not expect and the one that makes the portfolio finding non-obvious. Every decision tree we could fit over $(N, G, N/G, \text{topology})$ is beaten out of sample by the unconditional portfolio: the best reaches an exact-optimum rate of 0.564 (22 of 39) against 0.974, and its out-of-sample mean regret, 1.21–1.31, is worse even than the single-algorithm constant rule “always ParSub” (1.0797). A regime map over these axes is not merely unnecessary; it is actively harmful, because it fits partitions of a 39-instance suite whose cells hold two or three instances each. The simplest conditioner fails the same way: taking whichever algorithm was best on this instance’s topology in training attains the optimum on 21 of 39 held-out instances against 38, at mean regret 1.0488 against 1.0025. Topology is the axis on which conditioning is most intuitive, and the axis on which it most clearly does

not pay. Against the two pre-specified constant baselines the portfolio is significant on both confirmatory tests (McNemar $p = 0.0005$; Wilcoxon signed-rank on regret $p = 0.0003$; both survive Holm correction over the two comparisons).

It transfers to a topology the fit never saw. Leave-one-out holds out an instance but not its topology, and instances sharing a topology are not independent, so we also refit with an *entire topology* withheld and score each fold only on the topology it never saw. Across the six topologies the portfolio transfers better than conditioning generalises: mean out-of-sample regret 1.0391 (95% bootstrap interval [1.0025, 1.0885] over 2,000 resamples of the 39 instances), lower than the 1.0488 of a rule that conditions on topology and does see it in training. The exact measured optimum is still attained on 35 of 39 instances (89.7%), worst case 1.675. Withholding an entire leaf-count band instead leaves the leave-one-out figure unchanged (1.0025). We quote the interval and not the point estimate deliberately: with only six topologies the grouped estimate is genuinely uncertain at the top end, and a practitioner adopting the rule on an unseen topology should plan against 1.0885 rather than against the mean. The corresponding leave-one-out interval is [1.0000, 1.0076].

The mapping stage can be deleted. Assignment is the whole decision. Conditional on ParSub, the ten mapping algorithms give byte-identical cycle counts on all 39 instances. Fixing the mapping at Identity and choosing only the assignment attains the exact optimum on 38 of 39, at mean cost 0.046% and worst case 1.8%. We therefore recommend removing the mapping search from the pipeline rather than conditioning on it. Note what this does *not* say: Identity is not the best mapping — its apparent lead in the withdrawn winner table was a labelling artefact (§VI-E) — it is indistinguishable from the others and the cheapest to compute. In the other direction the spread across assignment algorithms is three orders of magnitude larger: Six of the eighteen are in the champion class on *zero* of 39 instances and cost a median $2.3\times$ to $13.1\times$ in cycles, up to $51.5\times$ at worst. Greedy list scheduling is the block to avoid.

What the rule does not license. Nothing here survives Holm correction over the 36 exploratory association tests we ran between algorithm families and instance parameters; two survive Benjamini–Hochberg, and we report the portfolio rather than any of them precisely because the portfolio needs no such association to hold. N and G are confounded across this suite by construction (Spearman $\rho = 0.871$, $p = 5.6 \times 10^{-13}$), so the budget rows of Table III that split on scale cannot be attributed to either variable separately. Of the 23 populated (size $\times$ topology) cells, 18 hold two instances or fewer, so no cell-level recommendation is derivable and we make none. The rule is untested at $G = 4096$, the one design point without a hardware result. And it is a retrospective statement about measured cycle counts, not a claim that the compiler can identify the winner from its own cost model. Every number is a measured cycle count on this pipeline for tree-structured reduction programs; none of it transfers to another compiler, another accelerator, or to MPI or NCCL collectives.

TABLE III

DERIVED ALGORITHM-SELECTION RULE. FOR EACH REGIME, THE S2 ASSIGNMENT ALGORITHMS A PRACTITIONER SHOULD RUN; THE S3 MAPPING MAY BE FIXED TO IDENTITY THROUGHOUT (SEC. VI-F). “OPT.” COUNTS INSTANCES ON WHICH THE RECOMMENDED SET ATTAINS THE HARDWARE-MEASURED MINIMUM CYCLE COUNT EXACTLY.

Regime	n	Run these S2 algorithms	Opt. Mean regret	Worst regret
Default (any N , G , topology)	39	DSC, Lukes, Frederickson, ParSub	38/39 1.003	1.099
restricted to non-ring	33	Lukes, Frederickson, ParSub	32/33 1.003	1.099
restricted to ring [†]	6	DSC, ParSub	4/6 1.019	1.063
Budget $k=1$: $N \geq 2048$	10	ParSub	7/10 1.094	1.607
Budget $k=1$: $N < 2048$, non-ring	24	Lukes	17/24 1.076	1.853
Budget $k=1$: $N < 2048$, ring [†]	5	DSC	2/5 1.092	1.300
Budget $k=2$ (any instance)	39	DSC, ParSub	19/39 1.030	1.220
Budget $k=3$ (any instance)	39	DSC, Lukes, ParSub	34/39 1.011	1.118

[†]Rests on six (resp. five) ring instances of which three are positive; reported for completeness, not recommended as a standalone rule. No (N -bucket $\times$ topology) cell in the suite holds more than three instances, so no cell-level rule is derivable.

VII. EXPORTING THE ABI: PINNED-FBT

Can the discipline be adopted without the SPRS fabric? It can, in software: replace a production collective’s dynamic G -dependent tree with a library that enforces the $\text{FBT}(N)$ post-order schedule whatever G and the topology are. We call this *pinned-FBT*. Two things must be said first, because we got the first wrong and the second is not ours.

The construction we published is incorrect. It partitioned canonical leaf indices into contiguous ranges, $\text{rank}(i) = \lfloor iG/N \rfloor$, and asserted that each rank then reduces exactly the subtree rooted at the lowest common ancestor of its leaves. Swept over all 44,850 pairs with $2 \leq N \leq 300$ and $2 \leq G \leq N$ that holds on 1,217 of them (2.71%); the smallest counterexample is $N = 3$, $G = 2$, where rank 0 owns leaves $\{0, 1\}$ whose lowest common ancestor is the whole tree’s root; and it fails on 8 of the 40 instances evaluated here. Nor is it merely a description error: evaluated literally in FP64 on our own leaf vector (5), the composed construction returns a root differing from $R_{\text{canonical}}(N, L)$ on 15,630 of those pairs, on 45.2% of the 1,546 pairs our own leaf counts admit, and on two of our own instances — $N = 10, G = 4$ by 2 ulp and $N = 200, G = 12$ by 1 ulp. The discrepancy is 1–3 ulp throughout, small in magnitude and total failure of a bitwise claim. A second, independent error survives any repair of the first: an all-reduce over sub-roots “consistent with $\text{FBT}(N)$ ’s top $\log_2 G$ levels” is $\text{FBT}(G)$ only when N and G are both powers of two. The published construction is exact precisely at power-of-two N and G .

The corrected construction. Cut $\text{FBT}(N)$ at depth $d = \lceil \log_2 G \rceil$ instead of partitioning index ranges. The cut frontier — nodes at depth d together with any leaf shallower than d — roots disjoint complete subtrees covering every leaf; assign those subtrees to the G ranks; each rank evaluates its subtrees by the canonical recurrence of Def. 1 under the enumeration of Def. 2; the cross-rank stage evaluates the internal nodes above the frontier, exactly $2^d - 1$ combines — the $\log_2 G$ -depth all-

reduce the published version wanted and did not have. The proof obligation is one sentence: every node of $\text{FBT}(N)$ is still computed exactly once, by the same recurrence and with the same operand binding (Def. 3), so the multiset of FADD operations and the dependency order among them are those of $\text{FBT}(N)$ itself and the root is $R_{\text{canonical}}(N, L)$ for every G . Enumeration discharges it over the same space that refutes the published version: every one of those 44,850 pairs is exact in FP64. The frontier-to-rank assignment does not affect the root — the tree fixes the order — so it is a free load-balancing choice, and the choice matters: under a greedy largest-first deal the worst leaf-count ratio is 1.993 and none exceeds 2, while under the natural contiguous deal it reaches 3.53 and exceeds 2 on 13,446 of them. An export must therefore *specify* the greedy deal.

Prior art, and the residue. TBIK [9] predates this manuscript; it is prior art, not concurrent work, and we claim no priority against it. Its Appendix C, Theorem 1 proves by induction, over the same contiguous-block partition, exactly the theorem our step 1 asserted without proof, under exactly the power-of-two hypotheses that make it true. The summary is uncomfortable and worth stating plainly: the correct part of pinned-FBT is what TBIK had already done, and the part TBIK did not do — arbitrary, non-power-of-two N — is the part that was wrong. What remains ours is the cut-aligned repair for arbitrary N under canonical heap-index leaf enumeration, together with the properties the export inherits from the ABI and TBIK does not test: invariance across network *topologies* rather than across tensor-parallel sizes inside one node, enforcement below the software stack, and presence-gated fail-stop.

Measured: the repair is invariant, and the checker can fail. We implemented both constructions on PyTorch’s collectives and ran them on a single node with two NVIDIA RTX PRO 6000 Blackwell GPUs over PCIe (no NVLink) and 72 CPU cores with NCCL on the two GPUs and the CPU backend for larger world sizes. Over 32 cases spanning N from 3 to 4,096 (ten of the leaf counts not powers of two — the case TBIK does not cover, and the majority of the sweep), world sizes 2–64 (seven not powers of two), messages from 8 KiB to 128 MiB, and two frontier deals, every one of the 2,414 (case, world size, rank) cells returned a single 64-bit root, equal to the single-process canonical $R_{\text{canonical}}(N, L)$. The result is reportable only because the same checker demonstrably fails: run against the index-range construction as a pre-registered negative control, three of four cases broke as they must — at $N = 1,000$ and $N = 1,500$ it returned three distinct roots across world sizes $\{2, 3, 8\}$ and *none* of its 13 cells equalled $R_{\text{canonical}}$. Where it agrees — ten of 13 cells at $N = 1,024$, all of them at $N = 7$ — it agrees by power-of-two coincidence.

Measured: what it costs, and the answer is unfavourable. The statistic was fixed before the run: geometric-mean overhead against stock `ncclAllReduce` at $G=2$ in FP64 over the swept message sizes, worst case beside it, with a support branch at roughly 1.0–1.3 $\times$ and an explicit unfavourable branch. The measurement took the unfavourable branch: over 20 configurations the geometric mean is 2.57 $\times$ stock, best 1.49 $\times$, worst 3.91 $\times$. Two qualifications pull in opposite

directions and neither changes the verdict. First, the sweep ran while an unrelated job saturated both GPUs, so the point estimate is *provisional*: substituting the best of the repeats at each configuration for the median gives $2.08\times$, so contention inflates the reported figure by about $1.24\times$. Both estimators are far outside the pre-registered support band, so the verdict does not depend on the correction — the point estimate does, and a re-run on a quiet machine is owed. Second, the overhead is not one quantity. At $G=2$ the pinned tree moves $2M$ bytes, one directed send of the full vector plus a broadcast, against a bandwidth-optimal ring's M per rank: a factor of about two is algorithmic and no engineering removes it. The remainder is an export artefact — two dependent library calls issued from Python against one fused NCCL kernel — dominating at small messages ($3.29\times$ at 8 KiB, where the whole stock call takes $57\ \mu\text{s}$) and fading at large ones ($1.49\text{--}1.67\times$ between 8 MiB and 32 MiB).

The result contradicts no claim this paper makes and prices one it already predicts. What we claim for pinned-FBT is a correctness property, measured above; what we do not claim is that it is cheap. It should not be set beside TBIK's 10%: that figure is four H20 GPUs over NVLink on an uncontended node, a different fabric measured a different way. On this evidence the export is usable where reshape reproducibility is the requirement and the collective is not the bottleneck, and not inside a bandwidth-critical loop. Closing the non-algorithmic part of the gap is an engineering problem; the algorithmic factor of two at $G=2$ is what the tree costs. Fabric-level and software-level enforcement can discharge the same structural obligation, but the record above is what it looks like when a construction is believed to discharge it and does not.

VIII. RELATED WORK

Relative to every axis here, what we claim is the conjunction of §I, not any single conjunct. *Reproducible arithmetic* — ReproBLAS [10], ExBLAS [11], Arteaga et al. [12] — obtains order-independence by changing the operator (pre-rounding, long accumulators, error-free transformations), so the result is reproducible but is not the FP64 sum any particular order produces, and a wide accumulator is required; compensated summation [13] reduces error without fixing order. We change no operator and add no accumulator, and fix the order instead. *Fixed-order schemes in deployed stacks* pin one axis inside one implementation: oneMKL's conditional numerical reproducibility is invariant to thread count and input partitioning [14]; TBIK [9] is invariant across tensor-parallel size and proves the power-of-two case of the construction §VII repairs, at a measured cost on NVLink far better than our software export's on PCIe; the “deterministic” modes of NCCL and RCCL hold within one (G , topology) and not across reshape [2], [15]. *In-network reduction* is the closest precedent for enforcing order in hardware, and we claim no advantage over it in the speed or cost of the reduction itself. SHARP [16] aggregates inside InfiniBand switches and pins operand order to do so, each aggregation node holding a predefined order of calculation [17]; its specification is equally explicit that reproducibility holds “for a given application-to-system mapping” — per deployment, where our index

is N and survives the deployment changing, which a per-mapping order cannot without standardising switch hierarchies across sites. It also prices the discipline structurally, a non-reproducible mode saving aggregation buffering by $n/2$ in the number of inputs. SwitchML [18] instead carries gradients in 32-bit fixed point through integer-only switch ALUs, so its order-insensitivity is integer associativity rather than a pinned order. Groq's TSP [19] shows a compiler/fabric determinism contract shipping, but the determinism is of the schedule: no bitwise claim is made, and its all-reduce is keyed to the packaging hierarchy, so reduction order is again a function of how many chips sit in a node. None states the invariant at a compiler/fabric interface indexed by problem size, enforced below the software that could violate it, with a fail-stop failure mode. *Collective-scheduling compilers* [1], [20]–[22] search schedule space for throughput, treating reduction order as free; that freedom is exactly what we remove. Our assignment, mapping, routing and pipeline components are all prior art [6]–[8]; the contribution is that any choice among them inherits invariance by composition.

IX. LIMITATIONS

L0 — One unbuildable configuration. Hardware validation covers 39 of the 40 instances. The exception is `maxp_mod_fat` ($N=8192$, $G=4096$, fat-tree), whose five attempted testbenches all hit the wall-clock cap for 58.9 core-hours — but wall-clock was not the binding constraint, and we correct an earlier account that said it was. The instance instantiates 4,192 routers while the adjacency interface carries a 12-bit router id whose top code is the disconnected sentinel, so every one of its 12,288 adjacency writes touches an id that aliases onto a low one. Replayed through the RTL's own truncation and range guards, the emitted configuration installs 8,223 links of which none match the intended fat-tree and delivers none of 900 sampled leaf-to-leaf routes, all 900 of which deliver on the untruncated fabric. No budget could have completed it. We have since repaired the truncation and re-elaborated: across three attempts under two `xelab` debug configurations, two ran elaboration to completion and exited zero reporting the snapshot built, and zero wrote a simulation kernel — so there is nothing for the simulator to load. We assert no root cause: a tool that exits zero having written no output is misreporting and we have not established why. The exclusion is a truncation defect in the emitter as shipped and, once repaired, a tool limit at this design point; it is neither a wall-clock budget nor a property of the fabric.

L0a — The denominator. On the 39 instances, 2,844 distinct images were attempted, 2,822 settled and passed, 22 did not settle. All but one of those are watchdog fail-stops — their per-PE counters stop at a last retirement plus exactly 20,000 cycles, far inside the testbench budget — and the remaining one is a simulator wall-clock kill. Non-settling runs are not evidence for Claim 1 and we do not count them as such; they are the campaign's only observation of Property 1 on unmutated compiler output. Coverage is narrower than the space we advertise: the 180 pairs enumerated per instance are 0.82% of the 21,960 triples, refinement is `None` in every

measured configuration, and the remaining axis is covered by Proposition 1 rather than by measurement.

L0b — Configuration-load bypass, measured. Route-table initialisation in every reported run is by hierarchical assignment at time zero rather than the clocked protocol, which costs $\mathcal{O}(G^2)$ simulated cycles. An earlier version attached an unreproducible fidelity band to that substitution; we have since measured it. Re-running every passing configuration at $G \leq 64$ under both paths gives the same verdict, root and cycle count on 1,326 of 1,326 points (100% bit-exact, no exceptions); the only effect of clocked loading is a deterministic prologue of one clock per route-table entry, completing before `program_start`; and a negative control confirms the clocked path genuinely exercises routing, corrupting the route table flipping `PASS` to `FAIL` on 24 of 24 runs. The residual is scope, not fidelity: the clocked path is uncharacterised above $G = 64$.

L1 — Scope. `fp64_add` is unpipelined. A stateless adder is what makes the arithmetic a pure function of its operand pair, which Claim 1 needs, but it is not a realistic synthesis target; we have run no synthesis and report no $F_{\max}$, slack or utilisation. Pipelining would change *when* a `COMPUTE` releases its scoreboard bit, not which operand pair it reads, so cycle counts would move and the roots in Table I would not. Beyond that: no silicon, no measured same-fabric overhead η (§V: a theorem on 31 of 39, unmeasured on the other 8), no head-to-head against production collectives on production interconnect, one leaf vector per N , one reduction per program (scoreboard bits are monotone within a program), and a suite whose (size $\times$ topology) cells are too thin for cell-level claims. FTZ+RNE is strictly narrower than IEEE 754-2019. C-sw is soft-enforced: a compile-time post-condition, not a fabric check. We prove no deadlock-freedom property, the emitted routes being minimal-distance rather than dimension-ordered. And Claim 1 has no formal proof: the argument is by code inspection and dynamic checking.

L13 — Two defect classes this evaluation cannot see. This one bounds the evaluation *method* rather than the artifact, and it is the item we would most want a reader building a similar campaign to take away. First, the campaign cannot exercise the FADD’s effective-subtraction path at all: Eq. (5) is strictly positive at every index, so both operands of every FADD in all 2,822 runs carry the same sign. The defect that lived on that path could not have appeared in any number reported here — not at this campaign size and not at any other, because the blindness does not lie along the axis the campaign enumerates: all 7,200 compilations share one leaf generator. Second, and closer to the bone, the oracle against which every root is compared is not an independent model of IEEE-754 addition. It is a transcription of the same adder and it reproduced the same defect bit for bit; on the as-submitted pair the two agree on all 62,320 vectors of the acceptance test while both disagree with correctly-rounded addition on 15,717 of them. A differential test is only as strong as the independence of its two sides, and ours were not independent. Both are cases of an evaluation that cannot detect a class of defect by *construction*, and neither is visible from inside the campaign: no pass rate, coverage figure or additional instance would have

exposed either, and both were found by reading the arithmetic against an external standard. Enumerating a parameter space exhaustively says nothing about the axes that space does not span, and the two axes that mattered here — operand sign, and the independence of the oracle — were not parameters of the campaign at all.

X. CONCLUSION

Cluster-reshape non-determinism in floating-point reductions is an *interface* problem, and the place to fix it is the contract between a collective-scheduling compiler and its fabric. Reduction order, treated as a size-indexed ABI over a canonical full binary tree, is orthogonal to G , topology, assignment, mapping, refinement and schedule, and yields cross-topology bitwise invariance: the 64-bit root is a pure function of N and the leaf vector. What we claim is a conjunction, not a first — that invariant, enforced at the compiler/fabric interface with a fail-stop failure mode, across network topologies rather than resource counts alone, over an unmodified IEEE-754 FP64 operator, for arbitrary non-power-of-two N .

On 40 instances spanning $N \in [8, 8192]$, $G \in [2, 4096]$ and six topologies, the campaign enumerated the refinement-free face of the compiler’s configuration space exhaustively and RTL-validated every distinct image it produced: of 2,844 attempted on 39 hardware-validated instances, 2,822 settled with a root bit-identical to the canonical one and 22 did not settle, all but one a watchdog fail-stop. Injected faults establish that the zero-divergence result is non-vacuous on 61–65 of 107 verdicts, the remainder being the halt outcome Property 1 predicts. We price the ABI structurally rather than against a non-deterministic baseline, which remains the principal open item in the evaluation.

Two negative results travel with the positive one. The compiler search this campaign was built around is not a ranking problem: the minimising configuration is a tie set on 38 of 39 instances, and what replaces the ranking — a fixed portfolio of 4 assignment algorithms reaching the measured optimum on 38 of 39 held-out instances, transferring to an entirely unseen topology at mean regret 1.0391 [1.0025, 1.0885] and beating every regime map we could fit — is both cheaper and better supported than the distribution it replaces. And two defect classes in our own artifact are invisible to this evaluation by construction: we found them by reading the arithmetic, not by running more configurations. An evaluation that enumerates a space exhaustively can still be blind along an axis that space does not span, and that blindness is not observable from inside the campaign.

Finally, *pinned-FBT* exports the discipline onto existing collective libraries. The construction we published is exact only at power-of-two N and G ; the cut-aligned repair given here is bitwise reshape-invariant on all 2,414 cells swept, against a pre-registered negative control that broke on three of four cases. Its cost is unfavourable and the statistic was fixed before the run: a geometric mean of $2.57 \times$ stock `ncclAllReduce` at $G=2$ in FP64, provisional under GPU contention, of which a factor of about two is the algorithmic price of the pinned tree

at two ranks. Reshape reproducibility is therefore available in software where the collective is not the bottleneck, and is the wrong purchase inside a bandwidth-critical loop. Fabric-level and software-level enforcement discharge the same structural obligation at very different prices; what makes either one checkable is that the reduction order is enforced rather than intended.

APPENDIX A THE FIFTEEN ASSUMPTIONS

Five compiler obligations, discharged by the compiler or the user; ten fabric invariants, established by RTL inspection. The four marked † are hypotheses of Property 1 only: violating one can stop a run but cannot alter a root the fabric does produce.

Compiler obligations. **C-sw** (single-writer allocation for remote destinations): partitioning each PE’s DMEM into *receive buffers* — addresses named as `target_addr` by a remote SEND — and *local temporaries*, the image contains for every PE p and receive buffer $a \neq 0$ exactly one write to (p, a) , and it is remote. Local temporaries are exempt: the allocator reuses them and their safety comes from in-order issue rather than single assignment. The scoping is load-bearing and the two classes are disjoint by construction; on the emitted images of 562 configurations the recycled and remotely-written address sets never intersect, and no operand is read after the write that recycles its slot (zero such events in 397,422). **C-bind** and **C-leaf** are Defs. 3 and 4, checked as post-conditions on the emitted image. **C-mem** (memory feasibility): every PE names at most `DMEM_DEPTH` = 512 distinct DMEM addresses and every emitted router id is below the adjacency sentinel (12 bits, at most 4,094). Neither is enforced at run time — the fabric truncates DMEM and SEND target addresses to ten bits, so an over-budget image aliases two addresses onto one slot silently, and out-of-range router ids alias onto low ids, which is what stopped one instance from ever settling (§IX, L0) — but both are cheap to discharge, peak occupancy being an $O(N)$ read-off the allocator already maintains. † **C-route**: the emitted tables encode destination-correct, loop-free paths. We claim those two properties only; the tables are minimal-distance rather than dimension-ordered, so we do not claim an acyclic channel-dependency graph.

Fabric invariants. **F-sb-mono**: for every receive buffer a , `scoreboard[a]` transitions $0 \rightarrow 1$ at most once per program, at the clock edge of the unique remote write, and stays set until reset. **F-sb-gate**: `COMPUTE` stalls `DE`→`EX` until both operand bits are 1, SEND until its source bit is. **F-fw**: a DMEM read at cycle t returns the value committed by the unique write at cycle $\leq t$, even across the adjacent-cycle race. **F-inorder**: each PE issues in order through `IF`→`DE`→`EX`, one instruction at a time, sampling both operands at the DE read port strictly before writing back in `EX` — which is what makes local-temporary reuse safe, since the scoreboard cannot, a recycled slot’s bit being already set. **F-gci-haz**: from `gci_start` until the GCI agent commits, DE stalls any `COMPUTE` or SEND naming the pending destination. **F-arb**: strict priority `PIU` > `GCI` > `FSM` with a GCI retry latch; no write is silently dropped. **F-fadd**: `fp64_add.sv` is combinational

and stateless, its output a pure function of the two 64-bit inputs sampled at the EX clock edge. † **F-credit**: per link and interval, $\#credit_returns \leq \#buffer_reads$ with bounded wait. † **F-misroute**: packets whose `dest_gpu_id` does not match the local PE are dropped, not silently consumed. † **F-watchdog**: a stall exceeding `WATCHDOG_MAX` = 20,000 cycles raises `P_ERROR`, as does an invalid opcode immediately.

REFERENCES

- [1] R. Rabenseifner, “Optimization of collective reduction operations,” in *ICCS 2004*, vol. 3036 of *LNCs*, pp. 1–9, Springer, 2004.
- [2] NVIDIA, “NCCL: Optimized primitives for collective multi-gpu communication,” <https://github.com/NVIDIA/nvcl>, 2018–2025.
- [3] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2nd ed., 2002.
- [4] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, “PyTorch distributed: Experiences on accelerating data parallel training,” in *VLDB*, 2020.
- [5] IEEE, “IEEE standard for floating-point arithmetic.” IEEE Std 754-2019 (Revision of IEEE 754-2008), 2019. doi: 10.1109/IEEESTD.2019.8766229.
- [6] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-effective and low-complexity task scheduling for heterogeneous computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
- [7] J. A. Lukes, “Efficient algorithm for the partitioning of trees,” *IBM Journal of Research and Development*, vol. 18, no. 3, pp. 217–224, 1974.
- [8] W. J. Dally and B. P. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA: Morgan Kaufmann, 2004.
- [9] Zhang, Ding, Yuan, Liu, Mao, Xing, and Liu, “Deterministic inference across tensor parallel sizes that eliminates training–inference mismatch,” arXiv:2511.17826, 2025.
- [10] J. Demmel and H. D. Nguyen, “Fast reproducible floating-point summation,” in *21st IEEE Symposium on Computer Arithmetic (ARITH)*, pp. 163–172, IEEE, 2013.
- [11] R. Iakymchuk, S. Collange, D. Defour, and S. Graillat, “ExBLAS: Reproducible and accurate BLAS library,” *Numerical Reproducibility at Exascale (NRE) Workshop*, SC, 2015.
- [12] A. Arteaga, O. Fuhrer, and T. Hoefer, “Designing bit-reproducible portable high-performance applications,” in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, IEEE, 2014.
- [13] W. Kahan, “Pracniques: Further remarks on reducing truncation errors,” in *Communications of the ACM*, vol. 8, p. 40, 1965.
- [14] Intel Corporation, “Conditional numerical reproducibility in the oneAPI Math Kernel Library.” Intel oneMKL Developer Guide.
- [15] AMD, “RCCL: ROCm collective communication library.” <https://github.com/ROCmSoftwarePlatform/rccl>, 2019–2025.
- [16] R. L. Graham, D. Bureddy, P. Lui, H. Rosenstock, G. Shainer, G. Bloch, D. Goldenberg, M. Dubman, S. Kotchubievsky, V. Koushnr, L. Levi, A. Margolin, T. Ronen, A. Shipiner, O. Wertheim, and E. Zahavi, “Scalable hierarchical aggregation protocol (SHArP): A hardware architecture for efficient data reduction,” in *COMHPC*, pp. 1–10, 2016.
- [17] Mellanox Technologies, “Aggregation protocol.” U.S. Patent 10,284,383 B2, 2019.
- [18] A. Sapio, M. Canini, C.-Y. Ho, J. Nelson, P. Kalnis, C. Kim, A. Krishnamurthy, M. Moshref, D. R. K. Ports, and P. Richtárik, “Scaling distributed machine learning with in-network aggregation,” in *NSDI*, 2021.
- [19] D. Abts *et al.*, “A software-defined tensor streaming multiprocessor for large-scale machine learning,” in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2022.
- [20] Z. Cai, Z. Liu, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, and O. Saarikivi, “Synthesizing optimal collective algorithms,” in *PPoPP*, 2021.
- [21] A. Shah, V. Chidambaram, M. Cowan, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, O. Saarikivi, and R. Singh, “TACCL: Guiding collective algorithm synthesis using communication sketches,” in *NSDI*, 2023.
- [22] R. Thakur, R. Rabenseifner, and W. Gropp, “Optimization of collective communication operations in MPICH,” *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005.